

Lecture Summary: Advanced OOP, Decorators, and File Handling

1. Access Control & Name Mangling Python uses underscores to signal how methods should be accessed:

- Protected Methods (`_method`): Accessible to the class and its subclasses. It is a "gentleman's agreement" that these shouldn't be called from outside.
- Private Methods (`__method`): These are meant to be strictly internal.
- Name Mangling: To prevent subclasses from accidentally overriding private members, Python renames `__method` to `_ClassName__method`. This makes it harder (but not impossible) to access from outside the class.

2. Abstraction (The Blueprint) Abstraction is used to define a common interface for a group of subclasses.

- ABC Module: Uses `from abc import ABC, abstractmethod`.
- Abstract Classes: You cannot create an object (instantiate) directly from an abstract class. It exists only to be inherited.
- Enforcement: Any subclass inheriting from an abstract class must implement all methods marked with `@abstractmethod`. This ensures that different classes (like `Square` and `Circle`) both have a required method (like `area()`).

3. Decorators Decorators are a powerful way to modify or "wrap" a function's behavior without changing its source code.

- Syntax: Applied using the `@decorator_name` symbol above a function.
- How they work: They take a function as an input, add some logic (like logging, authentication, or timing), and return a new version of that function.
- Use Case: Measuring how long a function takes to run (timing) or checking if a user is logged in before allowing a function to execute.

4. File Handling & Data Serialization The lecture transitions into how Python interacts with external data:

- **Basic Modes:** * **'r'**: Read (default).
 - **'w'**: Write (overwrites existing files).
 - **'a'**: Append (adds to the end of the file).
- **CSV Module:** Specifically designed for handling tabular data (Comma Separated Values).
- **JSON Module:** Used for "Serialization" (converting Python dictionaries/lists into a JSON string via **dump**) and "Deserialization" (converting JSON data back into Python objects via **load**).

5. Key Takeaways

- Mastering Name Mangling helps in writing secure, conflict-free classes.
- Abstraction standardizes how different parts of a large system interact.
- Decorators allow for cleaner, more modular code.
- File Handling is the bridge between a Python program and the real-world data it processes.
-